

XIAMEN
UNIVERSITY

COM102 – Programming Language C

Chapter 4 (iii) - Functions and Program Structure
Lab

Dr. Najla'a

Exercise 1

Vowel/Consonant Counter Using Functions

- Write a program that repeatedly asks the user to input a single character and counts how many vowels and consonants have been entered. The program should stop when the user enters the character '!’.
- Ignore digits, punctuation, whitespace, and special characters (only letters should be counted).

Exercise 1

```c
#include <stdio.h>
#include <ctype.h>

// Function to check if a character is a vowel
int isVowel(char c) {
    c = tolower(c);
    return (c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u');
}

int main() {
    char ch;
    int vowels = 0, consonants = 0;

    printf("Enter characters (type '!' to stop):\n");

    while (1) {
        printf("Enter a character: ");
        scanf(" %c", &ch); // space before %c to skip whitespace

        if (ch == '!')
            break;

        if (!isalpha(ch))
            continue;

        if (isVowel(ch))
            vowels++;
        else
            consonants++;
    }

    printf("\nTotal vowels: %d\n", vowels);
    printf("Total consonants: %d\n", consonants);

    return 0;
}
```

Exercise 2

Simple Calculator with Menu Function

- Write a C program that repeatedly displays a menu of mathematical operations and performs the selected operation on two numbers provided by the user. The program should stop when the user enters N or n.

Requirements:

- Create a **function named showMenu()** that displays the available operations:
  - A – Addition
  - S – Subtraction
  - M – Multiplication
  - D – Division
  - N – Exit
- Use a loop to:
  - Display the menu using showMenu()
  - Ask the user to select an operation (as a character)
  - If the operation is not N, ask for **two numbers**
  - Use a switch statement to perform the selected operation
  - Use if to handle invalid cases (e.g., division by zero)
- The menu must keep appearing until the user chooses to exit by entering N or n.

Exercise 2 - Solution

```c
#include <stdio.h>

void showMenu() {
    printf("\nA - Add\nS - Subtract\nM - Multiply\nD - Divide\nN - Exit\n");
}
int main() {
    char c;
    float a, b;
    do {
        showMenu();
        printf("Choice: ");
        scanf(" %c", &c);

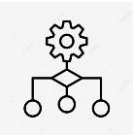
        if (c == 'N' || c == 'n') break;

        printf("First number: ");
        scanf("%f", &a);
        printf("Second number: ");
        scanf("%f", &b);

        switch (c) {
            case 'A': case 'a': printf("Result: %.2f\n", a + b); break;
            case 'S': case 's': printf("Result: %.2f\n", a - b); break;
            case 'M': case 'm': printf("Result: %.2f\n", a * b); break;
            case 'D': case 'd':
                if (b == 0) printf("Cannot divide by zero!\n");
                else printf("Result: %.2f\n", a / b);
                break;
            default: printf("Invalid choice.\n");
        }
    } while (1);

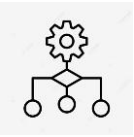
    printf("Goodbye!\n");
    return 0;
}
```

Exercise 3



1. Write a function: int max2(int x, int y) that **returns** the larger of x and y.
2. Write a function: int max3(int a, int b, int c); that must call max2 (do not re-implement comparisons directly) to return the largest of a, b, and c.
3. In main, call *max3(12, 5, 19)* and print only the result as a single integer on its own line.

Exercise 4



Integer power (recursion)

Do this:

1. Write `int ipow(int base, int exp)` that assumes *`exp >= 0`* and returns $base^{exp}$ using a loop.
2. Use the rule: $base^{0} = 1$, else *calculate* $base^{exp}$ using a loop
3. In main, print `ipow(2, 10)` (expected 1024).



Thank you

Q & A